

24CSA212 – Full Stack Frameworks

BUILDING RESTFUL APIS WITH EXPRESS JS

What is Express js?

- Express is a minimalist web framework that was highly inspired by the Sinatra framework from the Ruby language. With this module, you can create anything from small applications to large, complex ones. This framework allows you to build APIs and also to create simple web sites.

Express.js

Express.js is a minimal and flexible Node.js web application framework that provides a list of features for building web and mobile applications easily.

It simplifies the development of server-side applications by offering an easy-to-use API for routing, middleware, and HTTP utilities.

- Built on Node.js for fast and scalable server-side development.
- Simplifies routing and middleware handling for web applications.
- Supports building REST APIs, real-time applications, and single-page applications.
- Provides a lightweight structure for flexible and efficient server-side development.

Express.js



Built on Node.js



Routing and
Middleware

**Web Development
with Express JS**

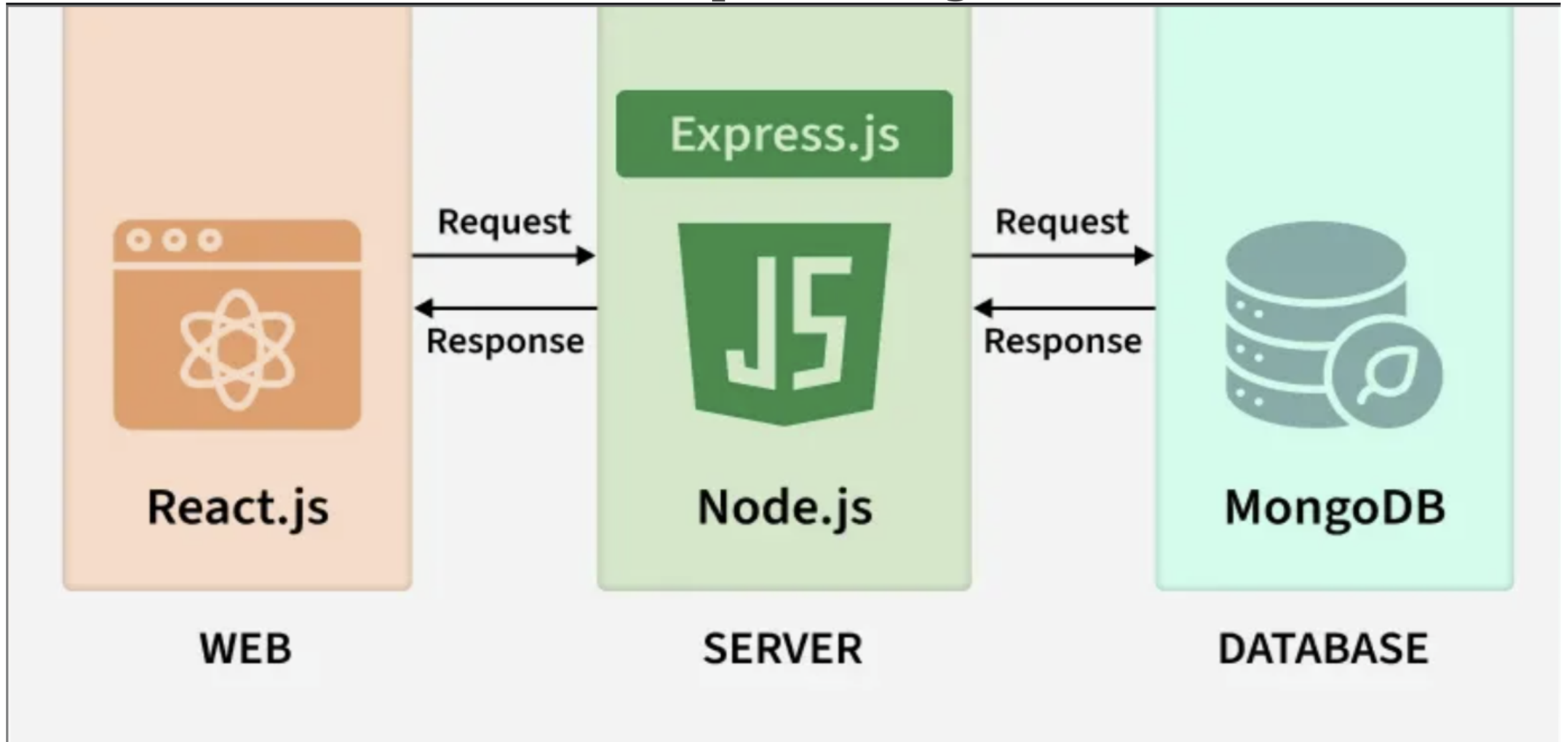


Full Stack
JavaScript



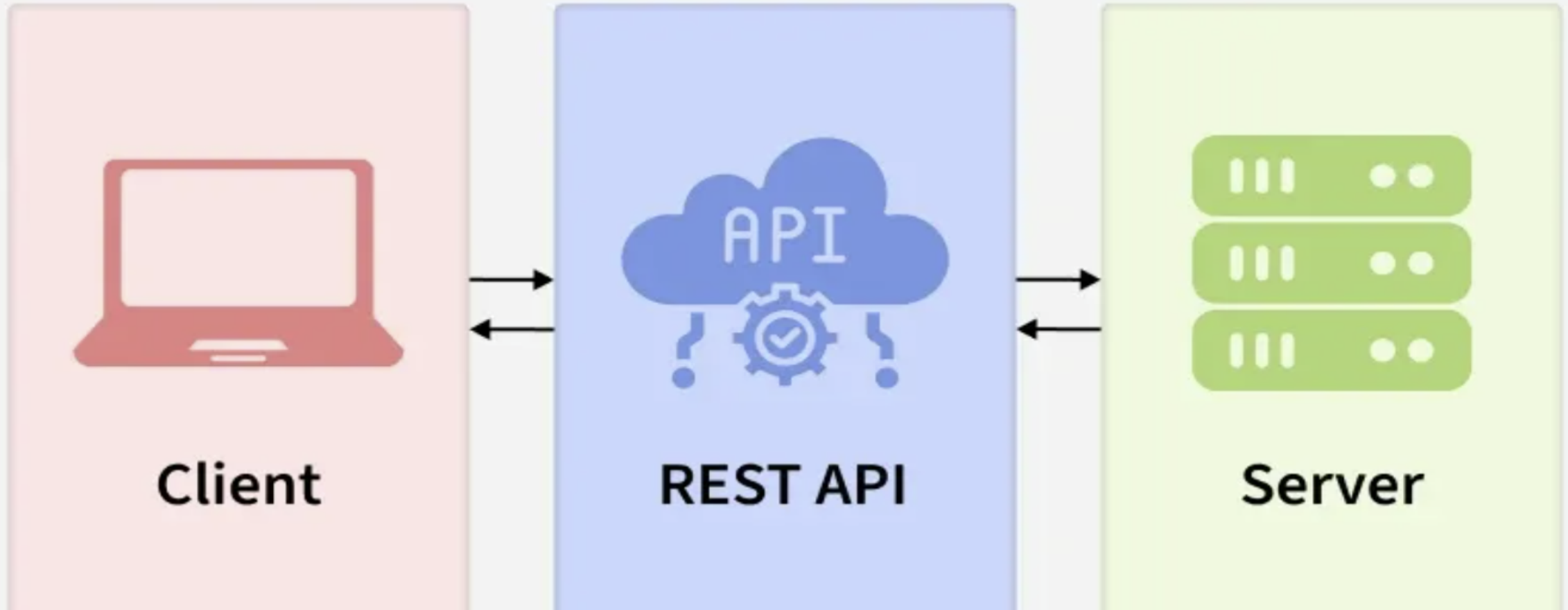
Restful APIs
Development

Express.js

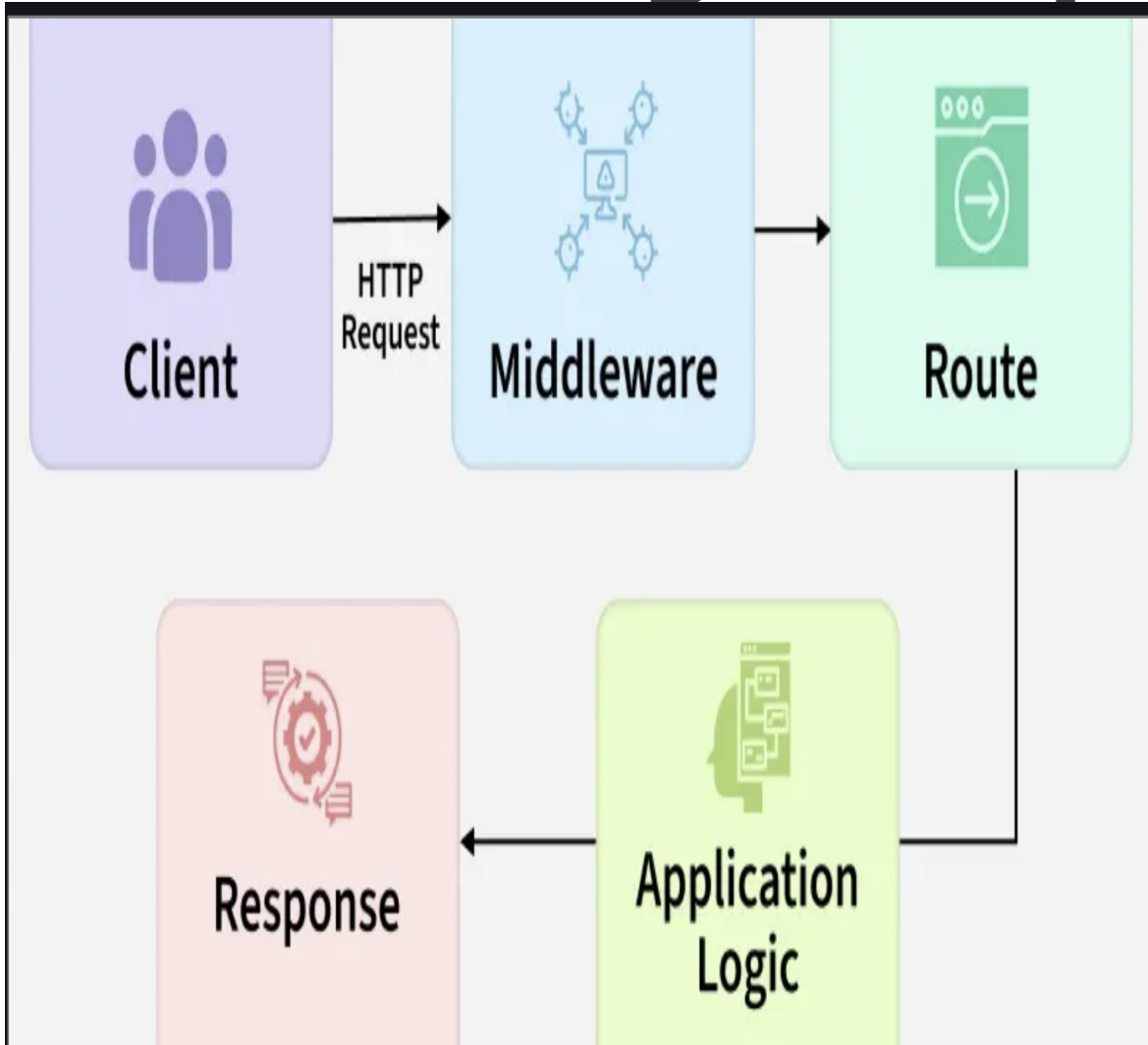


Express.js

Restful API in Express JS



Working of Express

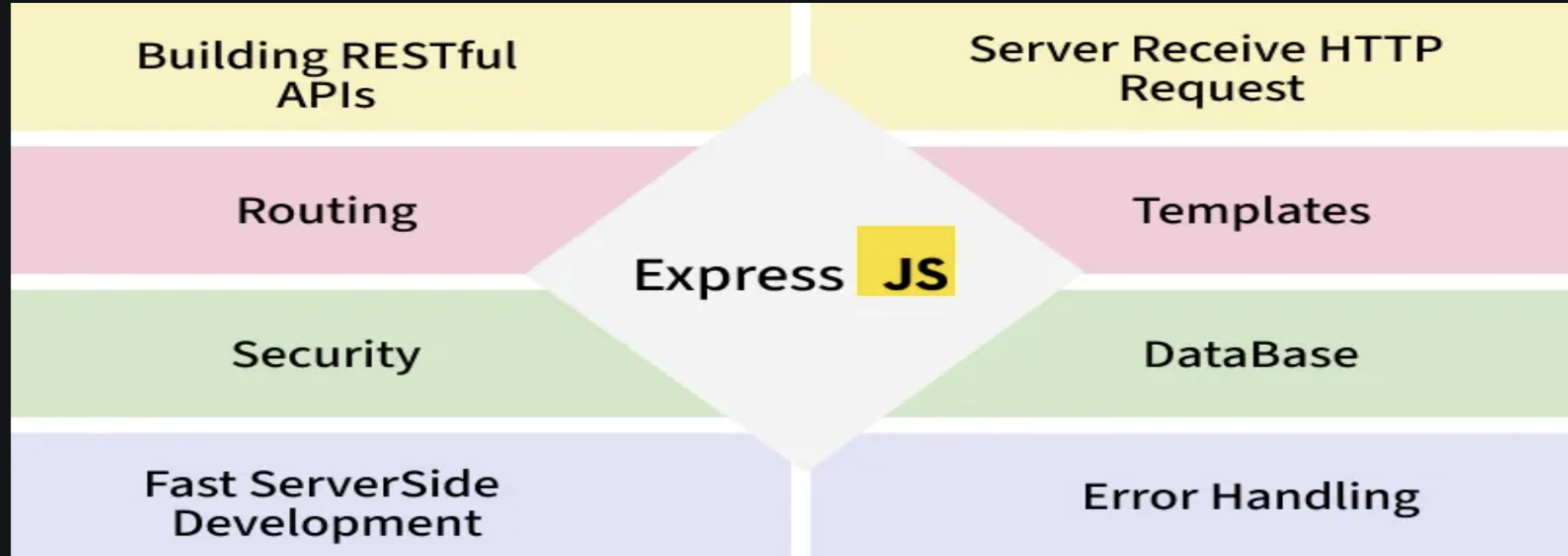


Express.js works by handling client requests through a series of steps called the request-response cycle, using middleware and routing to process requests and send responses efficiently.

- A client sends a request (e.g., browser or app) to the Express server.
- The request flows through middleware that handles logging, authentication, and other processing tasks.
- Express matches the request to a route handler based on the URL and HTTP method.
- The route handler processes the request and prepares a response.
- The server sends the response back to the client.
- If any error occurs, it is caught by error-handling middleware to ensure the server remains stable.

Express.js Features

Express.js is a minimal and flexible Node.js web application framework that simplifies building server-side applications and APIs.



- **Middleware-based:** Uses middleware to handle requests, responses, and common features.
- **Routing system:** Structure—developers can design apps as they want.
- **Integration:** Works well with databases (MongoDB, MySQL, PostgreSQL) and front-end frameworks.
- **Scalable:** Suitable for small apps to large-scale enterprise APIs.

Advantages of Express:

- 1. Easily integratable with template engines
- 2. flexible and Robust routing.
- 3. Minimalist code
- 4. A huge list of third-party modules available
- 5. Express is having biggest community available in nodejs

Install Express in a Node Project

Before adding Express to your project, ensure that NodeJS is installed on your system.

Setting Up Express

Step 1: Navigate to Your Project Directory

Open your terminal or command prompt and move to your project's root directory

```
cd path-to-your-project
```

Step 2: Initialize the NodeJS Project

Set up your project by creating a package.json file

```
npm init -y
```

This command initializes the project with default settings, creating a package.json file that manages your project's dependencies and metadata.

Step 3: Install ExpressJS

Add ExpressJS to your project

```
npm install express
```

This installs the ExpressJS framework and adds it to the dependencies section of your package.json.

Step 4: Create a Basic Express Server: In your project directory, create a file named app.js

First Express.js Program

```
// Import Express
```

```
const express = require('express');
```

```
const app = express();
```

```
// Define a route
```

```
app.get('/', (req, res) => {           //route to
```

```
rootURL
```

```
    res.send('Welcome to the Express.js ');
```

```
});
```

```
// Start the server
```

```
app.listen(3000, () => {
```

```
    console.log('Server is running on
```

```
http://localhost:3000');
```

```
});
```

```
    Welcome to the Express.js
```

In this example:

- Express is imported using `require('express')`, and an app instance is created with `express()`.

- A route is defined using the `app.get()` method, which responds with a message when the root URL (/) is accessed.

- The `app.listen()` method starts the server and listens on port 3000 for incoming requests.

It will start a server, and when you visit `http://localhost:3000`, it will display

Basic Routing in Express.js

Routing in Express.js is the process of mapping incoming HTTP requests (defined by method and URL) to specific handler functions. It enables developers to configure endpoints for various paths and operations, such as rendering views, processing form data, or performing CRUD actions on resources.

Basic routing in Express.js is the process of defining URL endpoints (routes) and specifying how the server should respond to client requests at those endpoints. Express provides simple methods to define routes that correspond to HTTP methods:

Syntax:

```
app.METHOD(PATH, HANDLER);
```

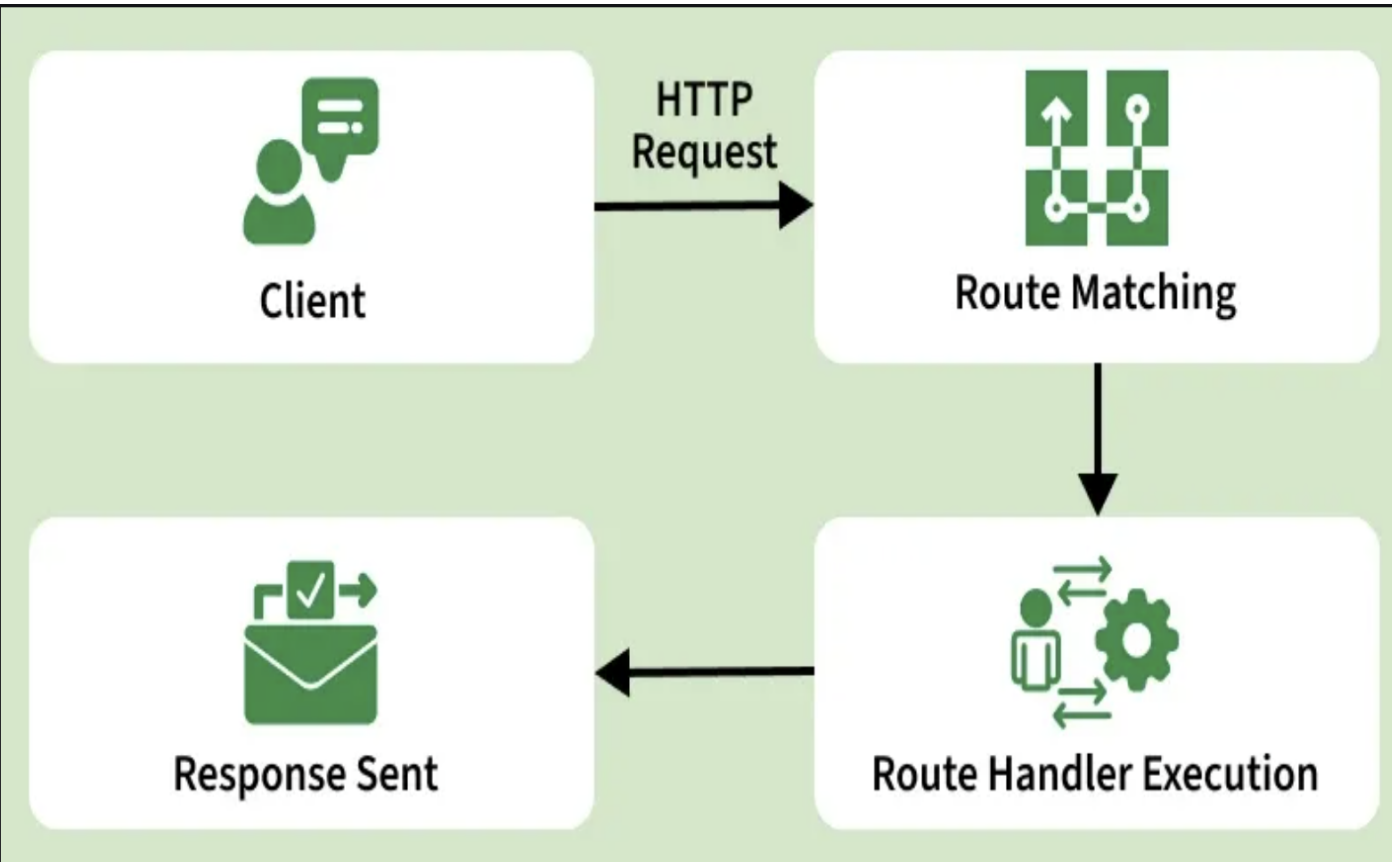
app: Represents an instance of an Express application.

METHOD: Represents an HTTP method like GET, POST, PUT, DELETE.

PATH: Defines the endpoint (route) where the request will be handled.

HANDLER: A function that executes when the route is accessed.

How Routing Work in ExpressJS?



Routing in Express.js follows a simple flow to handle client requests:

HTTP Request: Shows the incoming request with method and path (e.g., GET /users).

Route Matching: Lists all defined routes and shows which route matches the request (green check) and which don't (red cross).

Handler Execution: Indicates that the matching route's handler function is executed (req, res).

Sending Response: Shows how the server sends a response back to the client using methods like `res.send()`, `res.json()`, or `res.render()`.

Basic Routing in Express.js

```
const express = require('express');  
const app = express();  
const PORT = 4000;  
app.get('/', (req, res) => {  
  res.send('<h1>Welcome to the Home  
Page!</h1>');  
});  
app.get('/about', (req, res) => {  
  res.send('<h1>About Us</h1><p>This is  
the About page.</p>');  
})
```

```
app.get('/contact', (req, res) => {  
  res.send('<h1>Contact Us</h1><p>Feel  
free to reach out!</p>');  
});  
app.listen(PORT, () => {  
  console.log(`Server is listening at  
http://localhost:${PORT}`);  
});
```

<http://localhost:4000>
<http://localhost:4000/about>
<http://localhost:4000/contact>

Types of Routes

Basic Routes in ExpressJS

Basic routing involves defining a URL and specifying an HTTP method (GET, POST, PUT, DELETE, etc.).

```
app.get('/home', (req, res) => {    ///home handles GET requests.  
    res.send('Welcome to the Home Page!');  
});
```

```
app.post('/submit', (req, res) => {    ///submit handles POST requests.  
    res.send('Form Submitted Successfully!');  
});
```

Parameters in Express.js

Parameters in Express.js are values that are passed in the URL or request that your server can use to process the request dynamically.

They allow your routes to handle different inputs without creating multiple static routes.

Types of Parameters in Express:

- 1. Route Parameters:** allow capturing dynamic values from URLs, making routes flexible and reusable.

Part of the URL (e.g., /users/:id) accessed via req.params.

app.get('/users/:id', (req, res) => { *///
extracts the value.* *///:userId is a route parameter.* req.params.userId

```
const userId = req.params.id;  
res.send(`User ID is ${userId}`;  
});
```

Route Parameters continued

Optional and Multiple Route Parameters

Express allows defining optional parameters and multiple parameters in a single route.

```
app.get('/products/:productId?', (req, res) => {  
  res.send(`Product ID: ${req.params.productId} || 'No product selected'`);  
});
```

/products/123 → Product ID: 123

/products/ → No product selected

Multiple Route Parameters

```
app.get('/posts/:category/:postId', (req, res) => {  
  res.send(`Category: ${req.params.category}, Post ID: ${req.params.postId}`);  
});
```

/posts/tech/456 → Category: tech, Post ID: 456

Parameters in Express.js

2. Query Parameters: Appended to the URL (e.g., /search?term=nodejs) accessed via req.query.

```
app.get('/search', (req, res) => {  
  const searchTerm = req.query.term;  
  res.send(`Searching for ${searchTerm}`);  
});
```

3. Request Body Parameters: Sent in POST or PUT requests, accessed via req.body with express.json() middleware.

```
app.use(express.json());  
app.post('/users', (req, res) => {  
  const { name, age } = req.body;  
  res.send(`User created: ${name}, Age: ${age}`);  
});
```

Routing with parameters

```
const express=require('express');
```

```
const app=express();
```

```
app.get('/', function(req,res){  
  res.send('Hello World')  
}) // give the url as localhost:9000
```

```
app.get('/abc', function(req,res){  
  const id=req.query.id  
  res.send('Welcome back ABC' + id)  
}) //give the url as  
http://localhost:9000/abc?id=20
```

```
app.get('/abc/:id', function(req,res){  
  const id=req.params.id  
  res.send('Welcome ABC' + id)  
}) //give the url as  
http://localhost:9000/abc/:20
```

```
app.listen(9000, function(req,res){  
  console.log('Running...')  
});
```

Basic Routing in Express.js

- `app.get()` - Handle GET requests.
- `app.post()` - Handle POST requests.
- `app.put()` - Handle PUT requests.
- `app.delete()` - Handle DELETE requests.
- `app.all()` - Handle all HTTP methods.

Basic Routing in Express.js using HTTP methods

```
const express = require('express');  
const app = express();  
const PORT = 3000;
```

```
// Middleware to parse JSON bodies  
app.use(express.json());
```

```
// Basic Routing Examples
```

```
// Handle GET requests  
app.get('/get-example', (req, res) => {  
  res.send('This is a GET request');  
});
```

```
// Handle POST requests  
app.post('/post-example', (req, res) => {  
  res.send('This is a POST request');  
});
```

```
// Handle PUT requests  
app.put('/put-example', (req, res) => {  
  res.send('This is a PUT request');  
});
```

```
// Handle DELETE requests  
app.delete('/delete-example', (req, res) => {  
  res.send('This is a DELETE request');  
});
```

```
// Handle all HTTP methods  
app.all('/all-example', (req, res) => {  
  res.send(`This handles all HTTP methods: $  
    {req.method}`);  
});
```

```
// Start the server  
app.listen(PORT, () => {  
  console.log(`Server is running on http://localhost:$  
    {PORT}`);  
});
```


Middleware in Node.js

- Middleware in NodeJS refers to a software design pattern where functions are invoked sequentially in a pipeline to handle requests and responses in web applications.
- It acts as an intermediary layer between the client and the server, allowing for modularization of request processing logic and enabling cross-cutting concerns such as authentication, logging, error handling, and data transformation.

Middleware Functions:

- In NodeJS, middleware functions are JavaScript functions that have access to the request object (req), response object (res), and the next middleware function in the pipeline (next).
- Middleware functions can perform tasks such as modifying request or response objects, terminating the request-response cycle, or passing control to the next middleware function in the pipeline.

Middleware in Express.js

- Express.js, a popular web framework for NodeJS, heavily utilizes middleware for request processing.
- Middleware functions in Express.js can be added using the `app.use()` method or specific HTTP method functions such as `app.get()`, `app.post()`, etc., to specify middleware for specific routes.
- Middleware functions can perform various tasks such as executing code, modifying the request and response objects, ending the request-response cycle, or calling the next middleware function in the stack.

Types of Middleware

Express.js offers different types of middleware, each serving a specific purpose:

Application-level middleware: This middleware is bound to an instance of the app object using `app.use()` or `app.METHOD()`. It can be used for tasks such as logging, authentication, and handling errors. For example:

```
const express = require('express');  
const app = express();
```

```
app.use((req, res, next) => {  
  console.log('Time:', Date.now());  
  next();  
});
```

```
app.get('/user/:id', (req, res, next) => {  
  res.send('USER');  
});
```

```
app.listen(3000);
```

Types of Middleware

Router-level middleware: This middleware works similarly to application-level middleware but is bound to an instance of `express.Router()`.

It is useful for modularizing your application by creating separate routers for different parts of your app. For example:

```
const express = require('express');  
const app = express();  
const router = express.Router();
```

```
router.use((req, res, next) => {  
  console.log('Time:', Date.now());  
  next();  
});
```

```
router.get('/user/:id', (req, res, next) => {  
  res.send('User Info');  
});
```

```
app.use('/', router);
```

```
app.listen(3000);
```

Types of Middleware

Error-handling middleware: This middleware is defined with four arguments (err, req, res, next) and is used to handle errors during the request-response cycle.

For example:

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});
```

Middleware in Express.js

```
const express = require('express');  
const app = express();  
const PORT = 3000;  
  
// Middleware to log requests  
app.use((req, res, next) => {  
  console.log(`${req.method} request for $  
    {req.url}`);  
  next(); // Pass control to the next  
middleware/route  
});
```

```
// Middleware to parse JSON bodies  
app.use(express.json());  
  
// Route handler  
app.get('/', (req, res) => {  
  res.send('Hello, Express with middleware!');  
});  
  
app.listen(PORT, () => {  
  console.log(`Server running on  
    http://localhost:${PORT}`);  
});
```

Types of Middleware

Built-in middleware: Express provides some built-in middleware functions such as `express.static` for serving static assets, `express.json` for parsing JSON payloads, and `express.urlencoded` for parsing URL-encoded payloads.

Third-party middleware: You can use third-party middleware to add functionality to your Express apps. For example,

```
//to use the cookie-parser middleware:  
const cookieParser = require('cookie-parser');  
app.use(cookieParser());
```

Middleware - Conclusion

Middleware is a powerful and flexible feature of Express.js that allows you to handle requests and responses in a modular and reusable way.

By using different types of middleware, you can add various functionalities to your application, such as logging, authentication, error handling, and more.

Error Handling in Express.js

Error Handling Middleware:

Middleware functions can also handle errors by defining error-handling middleware using four parameters (err, req, res, next). Error-handling middleware functions are invoked when an error occurs during request processing.

```
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to simulate an error
app.get('/error', (req, res, next) => {
  const err = new Error('Something went wrong!');
  err.status = 500;
  next(err); // Pass error to error-handling middleware
});

// Error-handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack); // Log the error
  res.status(err.status || 500).send({
    message: err.message,
    status: err.status || 500
  });
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Error Handling in Express.js

Error handling in Express refers to the process of managing errors that occur during the execution of a request.

When an error occurs in a route or middleware, Express provides mechanisms to catch these errors and prevent the application from crashing.

Error-handling middleware allows developers to:

- Log errors for debugging.
- Send structured error responses to users.
- Maintain application stability even when issues arise.
- Good error handling makes an application more reliable and user-friendly, improving both developer efficiency and the user experience.

How Error Handling Works

Express ensures errors are caught and managed effectively to prevent application crashes.

- Error Detection: Automatically detects issues, such as bad requests or server failures.
- Error-Handling Middleware: Uses special middleware to catch and process errors in a structured way.
- Passing Errors: Errors in routes are passed to the middleware using `next(err)`.
- Logging: Errors are logged to help developers quickly identify and resolve issues.
- User-Friendly Responses: Express sends clear, non-technical messages to users instead of showing stack traces.

Types of Error Handling in Express

There are multiple ways to handle errors in Express;

1. Default Error Handler

Express has a built-in error handler that catches any errors that are not handled. If you don't create your own error handler, Express will automatically send a simple "500 Internal Server Error" response.

```
app.get("/", (req, res) => {  
  throw new Error("Something went wrong!");  
});
```

```
Error: Something went wrong!  
    at C:\Users\GFG0569\Pictures\Screenshots\error\server.js:5:9  
    at Layer.handle [as handle_request] (C:\Users\GFG0569\Pictures\Scr  
    at next (C:\Users\GFG0569\Pictures\Screenshots\error\node_modules\  
    at Route.dispatch (C:\Users\GFG0569\Pictures\Screenshots\error\nod  
    at Layer.handle [as handle_request] (C:\Users\GFG0569\Pictures\Scr  
    at C:\Users\GFG0569\Pictures\Screenshots\error\node_modules\expres  
    at Function.process_params (C:\Users\GFG0569\Pictures\Screenshots\  
    at next (C:\Users\GFG0569\Pictures\Screenshots\error\node_modules\  
    at expressInit (C:\Users\GFG0569\Pictures\Screenshots\error\node_m  
    at Layer.handle [as handle_request] (C:\Users\GFG0569\Pictures\Scr
```

- Express automatically catches errors that are not handled.
- It prevents the server from crashing when unexpected issues occur.
- If you don't add your own error handler, Express uses its default one.
- In development mode, the error message is shown, but in production, it is hidden.

Types of Error Handling in Express

2. Custom Error-Handling Middleware

Instead of relying on the built-in error handler, you can create a custom error middleware. This lets you log errors and send user-friendly responses.

```
// Custom error-handling middleware
```

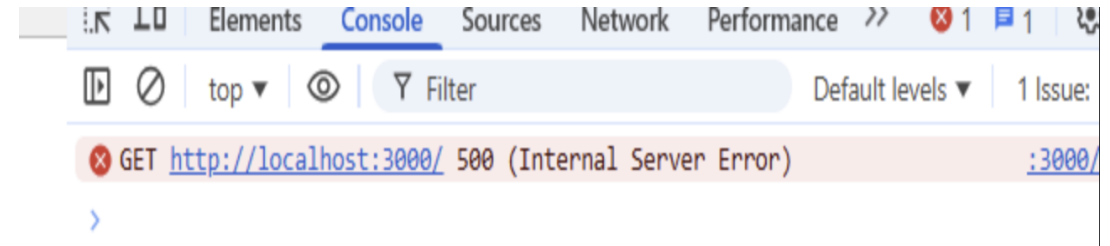
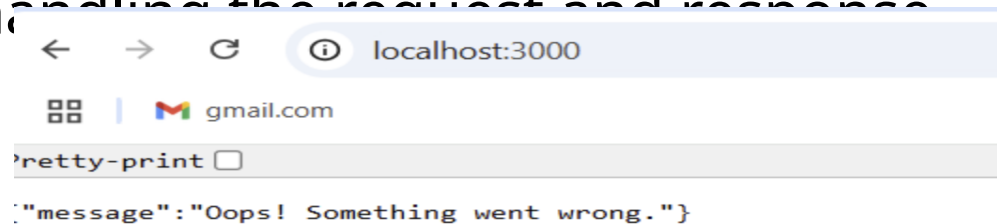
```
app.use((err, req, res, next) => {
```

```
  console.log(err);
```

```
  res.status(500).json({ message: "Oops! Something went wrong." });
```

```
});
```

- `app.use((err, req, res, next))`: This middleware runs only if an error occurs in a route. It catches the error, logs it, and sends a structured JSON response to the client.
- The `err` object contains the error details, while the `req` and `res` objects are used for handling the request and response.



Types of Error Handling in Express

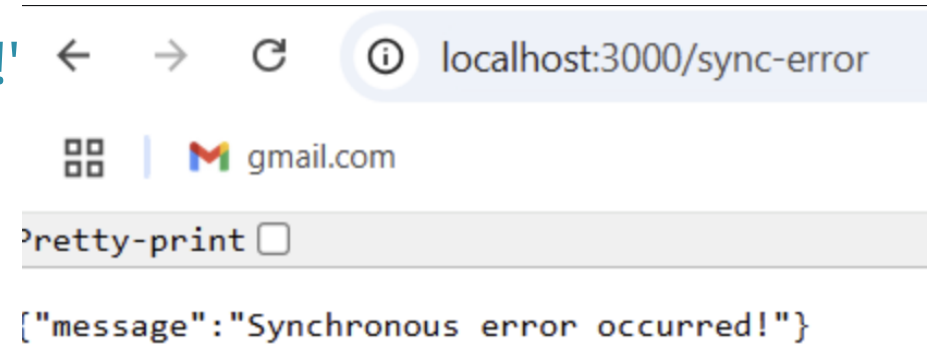
3. Synchronous Error Handling

Express automatically catches errors in synchronous functions. If an error is thrown inside a route, it is sent to the error handler.

```
app.get("/sync-error", (req, res) => {  
  throw new Error("Synchronous error occurred!")  
});
```

```
// Custom error handler  
app.use((err, req, res, next) => {  
  res.status(500).json({ message: err.message });  
});
```

- Any error thrown in the route (throw new Error) is caught by Express and forwarded to the error-handling middleware automatically.
- The custom error handler sends the error message as a JSON response with a 500 status code.



Types of Error Handling in Express

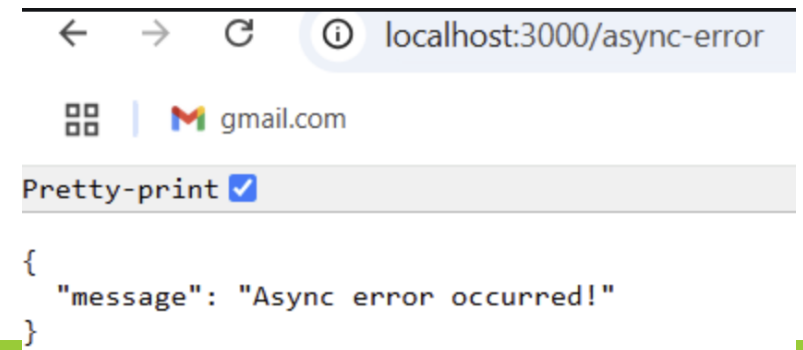
4. Asynchronous Error Handling

Unlike errors in normal functions, errors in async functions are not caught automatically by Express. You need to handle them manually using try-catch or pass them to Express using `next(err)`.

```
const express = require("express");  
const app = express();  
  
// Error handler middleware  
app.get("/async-error", async (req, res, next) => {  
  try {  
    await Promise.reject(new Error("Async error occurred!"));  
  } catch (err) {  
    next(err);  
  }  
});  
  
app.use((err, req, res, next) => {  
  res.status(500).json({ message: err.message });  
});  
  
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

In this code, the error is manually caught in the catch block and passed to the error handler using `next(err)`.

The error-handling middleware then responds with a 500 status code and the error message.



Types of Error Handling in

5. Handling Errors with next(err) Express

Calling next(err) inside a route manually forwards errors to Express's error handler.

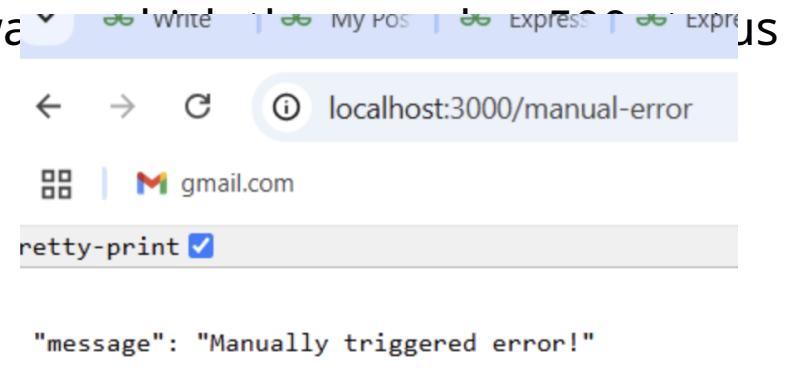
```
const express = require("express");
const app = express();

app.get("/manual-error", (req, res, next) => {
  const err = new Error("Manually triggered error!");
  next(err);
});
```

- When someone visits /manual-error, an error is manually created and passed to Express using next(err).
- The error is automatically sent to the error-handling middleware with the error message.

```
app.use((err, req, res, next) => {
  res.status(500).json({ message: err.message });
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```



Usage of Express.js

Express.js is a popular web framework for Node.js that simplifies server-side development. It is widely used to build web applications, APIs, and scalable server-side solutions.

- **Web Applications:** Build dynamic sites and SPAs efficiently.
- **RESTful APIs:** Create APIs for CRUD operations and data delivery.
- **Middleware Apps:** Add logging, authentication, and validation.
- **Microservices:** Lightweight and modular for scalable services.
- **Server-Side Rendering:** Supports EJS, Pug, Handlebars.
- **Proxy/Gateway:** Routes and manages HTTP traffic.
- **Rapid Prototyping:** Quickly develop and test server logic.